Team Name : KALI

Team Leader Name : P Yashwanth Reddy

Problem Statement : Intelligent Candidate Discovery & Ranking Challenge

Solution Overview

Proposed Solution: KALI (Knowledge-Augmented Listing Intelligence)

- Built a pure-python multi-factor weighted scoring algorithm that evaluates candidates across 6 core dimensions.
- Implemented zero-dependency deterministic logic running entirely in memory, processing 100K candidates in 10 seconds on CPU.
- Incorporates a multi-indicator honeypot detection filter and keyword-stuffer penalty system.
- Generates unique, grounded, non-templated reasoning for each candidate in the top 100.

JD Understanding & Candidate Evaluation

Key Requirements Extracted from JD:

- Experience: 5–9 years (sweet spot for founding team senior engineer).
- Core Tech: Embeddings-based retrieval (sentence-transformers), vector DBs (FAISS, Milvus, Pinecone), strong Python.
- Background: Experience at product companies (services/consulting giants are penalized).
- Logistics: India-based (Pune/Noida preferred or willing to relocate), short notice period (<30 days preferred).

Ranking Methodology

Ranking Methodology & Multi-Factor Scoring:

- Phase 1 Cascade Filter: Fast elimination pass (YoE, inactivity, keyword stuffers, and lack of career ML keywords) — filters out ~60% of candidates in milliseconds.
- Title & Career Relevance (35%): Matches taxonomy of target ML/AI titles and scans career history descriptions for AI keywords.
- Skills Match (25%): Computes a weighted sum of AI skills, adjusted by proficiency level, usage duration, and endorsements.
- Experience Fit (15%): Optimal bell curve centered on 7 years, with penalties for job hopping.
- Behavioral Signals (15%): Response rate, profile completeness, GitHub activity, interview attendance, and reply speed.
- Location & Notice (5%): India location, Pune/Noida match, and notice period length.
- Education (5%): Field of study relevance, degree level, and university tiering.

Multipliers applied: Honeypot (0.0x), Keyword Stuffer (0.05x), Services-Only (0.85x), No Career ML keywords (0.10x).

Explainability & Data Validation

Explainability, Quality Control & Validation:

• 100% Grounded Justifications: A dynamic reasoning generator constructs sentences based on the candidate's exact title, years of experience, specific skills matched, current company, and location. Hallucinations are physically impossible because we only inject fields from the candidate's record.
• Zero Template Duplication: Avoids duplicate reasoning strings by hash-varying sentence patterns and location/notice/activity phrases based on candidate ID.
• Anomaly/Honeypot Filter: Profiles claiming 'expert' in many skills with 0 months duration, or years of experience inconsistent with career dates, are filtered out and assigned a score of 0.

End-to-End Workflow

End-to-End Execution Workflow (3-Phase Cascade):

1. Read Input: Stream candidate records line-by-line from candidates.jsonl.gz (memory-efficient).
2. Phase 1 Elimination Filter: Reject honeypots, inactive profiles (>240d), keyword stuffers, YoE outside [3, 15], and profiles lacking career ML keywords. (Eliminates ~60% of candidates instantly).
3. Multi-Factor Scoring: Calculate scores for Title, Skills, Experience, Behavioral, Location, and Education for remaining candidates.
4. Apply Penalty Multipliers: Demote stuffers, services-only candidates, inactive profiles, and those lacking career keywords.
5. Sort & Rank: Sort candidates descending by score, tiebreaking on Candidate ID ascending.
6. Generate Reasonings: Run dynamic, grounded text generation for the top 100.
7. Export: Write the ranked records to the validated CSV format.

System Architecture — KALI Pipeline:

[candidates.jsonl.gz]

↓

[Honeypot Anomaly Filter] → Score = 0 (If Flagged)

↓

[Multi-Factor Scorer]

Title (35%) • Skills (25%) • Exp (15%)

Signals (15%) • Loc (5%) • Edu (5%)

↓

[Penalty Adjustments]

Stuffers (0.05x) • Services-only (0.85x)

Inactive (0.4x) • No Career ML (0.1x)

↓

[Sorting & Tie-Breaking]

Results & Performance

Results, Performance & Insights:

- Execution Speed: Processed 100K candidates with Phase 1 elimination in 10.7 seconds. Peak RAM: 60MB. CPU cores utilized: 1.
- Official Format Verification: Passed the official validate_submission.py check with 100% compliance.
- Deep Quality Validation Metrics:
  - Honeypot Rate: 0% in Top 100 (Disqualification threshold is >10%).
  - Keyword Stuffer Rate: 0% in Top 100.
  - Career History ML Relevance: 100% in Top 10 and Top 50 candidates.
  - Reasoning Grounding & Uniqueness: 100% grounded in profile facts; 0 duplicates.
  - Behavioral Signals: 100% active, highly responsive profiles in Top 10.

Technologies Used

Technology Stack & Rationale:

1. Python 3.10 (Standard Library): Used for the core ranking engine. Standard modules (json, csv, gzip, re, datetime, math) guarantee zero package installation errors, fast startup, and high predictability.
2. Gradio: Selected for the HuggingFace Spaces sandbox deployment. Provides an intuitive, responsive web UI for uploading candidate segments and downloading ranked CSVs in real time.
3. python-docx: Used offline to parse the Job Description and Redrob reference documents to engineer features.
4. Git & GitHub LFS: Used to manage versioning and track the gzipped candidate dataset (54MB) cleanly.

Submission Assets & Links:

- GitHub Repository:
  https://github.com/yashreddy1154/intelligent-candidate-ranker
  Contains the fully reproducible ranking script, README instructions, and validation suite.

- HuggingFace Spaces Web Sandbox:
  https://huggingface.co/spaces/yash1154/intelligent-candidate-ranker
  A working demo to upload candidate subsets and run matching on the fly.

- Final Submission File:
  submission.csv (UTF-8, containing exactly 100 ranked candidates with grounded reasoning).

redrob | H2S

INDIA.RUNS

Build what next India runs on

THANK YOU